

GNR-638 : Assignment 2 report

Sarvesh Shashidhar (24M2152), Anirban Saha (25D1598), Yash Sarang (24M2160)

March 16, 2026

Introduction

This report presents a systematic and brief study of the experiments done as a part as the **second assignment** under the GNR-638 course structure. Through this study, we explore representation transfer, data efficiency, and robustness of pre-trained CNN backbones. We evaluate **three** ImageNet-pretrained architectures–

1. ResNet-50
2. ConvNeXt-Tiny
3. DenseNet-121

The dataset used for evaluation is the *Aerial Image Dataset (AID)* for multi-class scene classification. The study compares these models on this dataset using **five** different experimental set-ups –

1. Linear probing
2. Fine-tuning strategies
3. Few-shot learning
4. Robustness against image corruptions.
5. Layer-wise representation quality.

Through these experiments, we analyze the transferability and robustness characteristics of the CNN architectures and highlight how architectural design influences the performance.

Dataset Properties

The dataset has **30 classes** with approximately *250 images* per class. Each image has a dimension of 600×600 , but these were **re-sized** to 224×224 for better handling by the CNN models. This also ensured uniformity within the data.

Data preprocessing

The data was pre-processed by performing three operations -

1. First, the images were re-sized to be 224×224 as mentioned above.
2. Then, the training images were **augmented** by performing *horizontal flipping*. This ensured that the models do not overfit and generalize well.
3. Finally, the images underwent the standard ImageNet normalization

These pre-processing steps were done for both training and validation sets, but for the validation set we refrained from flipping the image horizontally to preserve the true nature of the image.

Experimental Setup

With the data pre-processed and ready, we proceeded with the experimentation. All the models were initialized with ImageNet pre-trained weights using the `timm` library. These models were then evaluated via **five** experiments which shall be highlighted later in this document. For each model, we passed the parameters as shown in Table 1.

Throughout this assignment, we used the **ADAM** optimizer and the parameters as mentioned in Table 1 remained consistent (until and unless mentioned otherwise) across models to ensure uniformity in analysis. We also observed the model efficiencies as highlighted in Table 2. With these values noted and the experimental setup formalized, we performed the different evaluations as expected from the assignment.

Table 1: Parameter and their values used for experimentations

Parameter	Description	Value used
DATA_DIR	Relative path to the data directory	../train_data
EPOCHS	Training epochs	30
SMALL_EPOCHS	Epochs for few-shot analysis	20
BATCH_SIZE	Training batch size	1024
SUBSET_BATCH_SIZE	Batch size for few-shot and layer-wise learning	64
PLOT_PATH	Path to store plots	../Figures/<model_name>/
LR	Learning rate	$1e^{-3}$
SEED	Seed for reproducibility	42

Table 2: Model complexity comparison

Model	Parameters (M)	MACs (G)	FLOPs (G)
ResNet50	23.51	4.132	8.263
DenseNet121	6.87	2.833	5.667
ConvNeXt-Tiny	27.8	4.455	8.91

Experiment 1 – Linear Probe Transfer

In this experiment, the backbone parameters are *frozen* and the final classification layer is replaced with a new classifier that is trained for the data at hand. We have only trained the linear classifier and not the entire network. The classifier was trained for **30 epochs** over the training data and the accuracy per epoch was plotted as shown in Figure 1a

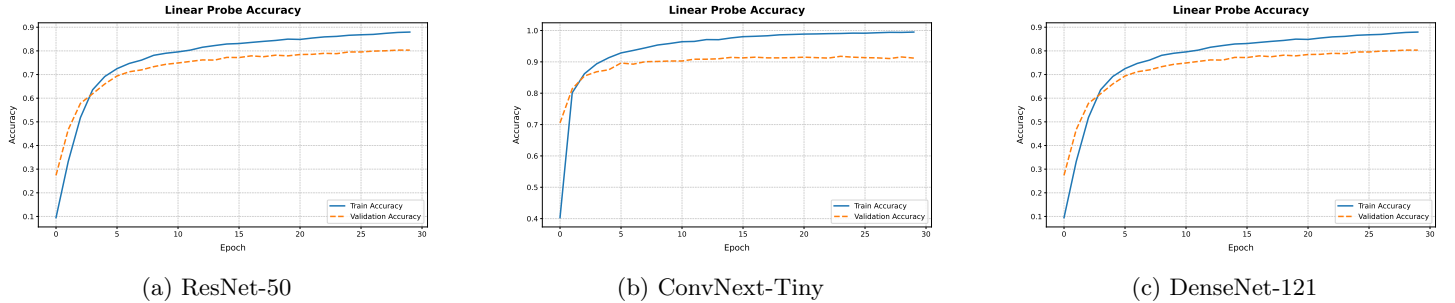


Figure 1: The variation of training and validation accuracy with epochs for the three CNN backbones.

The accuracy and correctness of the model can also be observed by plotting the **confusion matrix** as shown in Figure 2. To visualize the features learnt by the classifier, **t-SNE** and **PCA** plots were created as shown in Figure 3 and Figure 4 respectively.

Experiment 2 – Fine-Tuning Strategies

We evaluated how different fine-tuning strategies affect model performance and stability. The following strategies were used -

- Linear probe
- Last block fine-tuning
- Full fine-tuning of the entire backbone
- Selective layer unfreezing (20% of the parameters were unfrozen)

Rest of the hyperparameters were kept the same to maintain uniformity. The first method to check the performance of the models was to check the training and validation accuracy as the percentage of the unfrozen parameters change. First, we take the accuracy variation for **linear probe** where only the classifier is trainable. The plots are shown in Figure 5

Then, we perform the same analysis by freezing the last block as well as the classifier parameters, results of which have been plotted in Figure 6. We then performed *full fine-tuning* which is essentially when we unfreeze the entire CNN backbone. The accuracy variation of this case is shown in Figure 7. Finally, we checked the accuracy values when only **20%** of the parameters

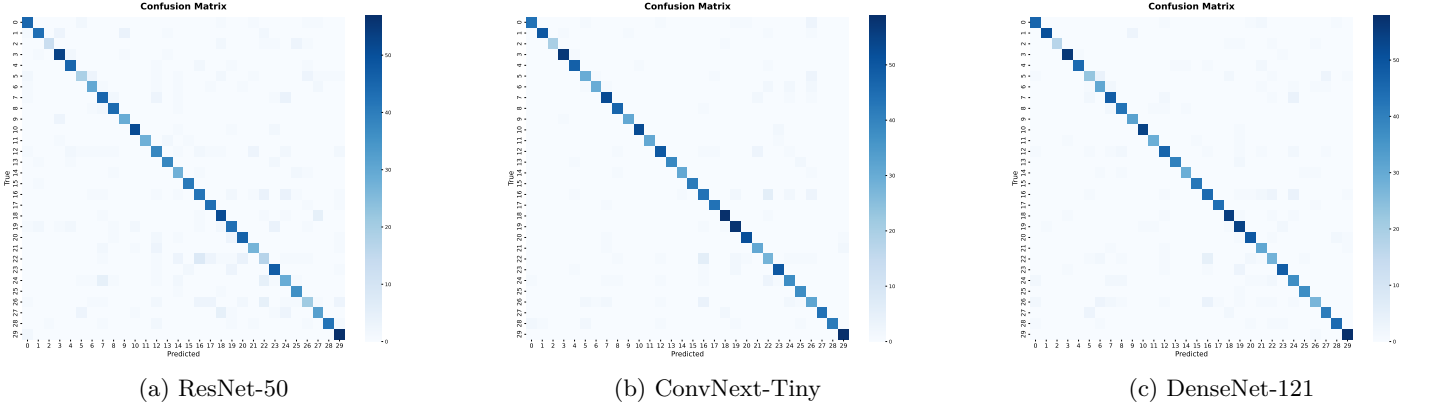


Figure 2: Confusion matrices for the classifiers after training.

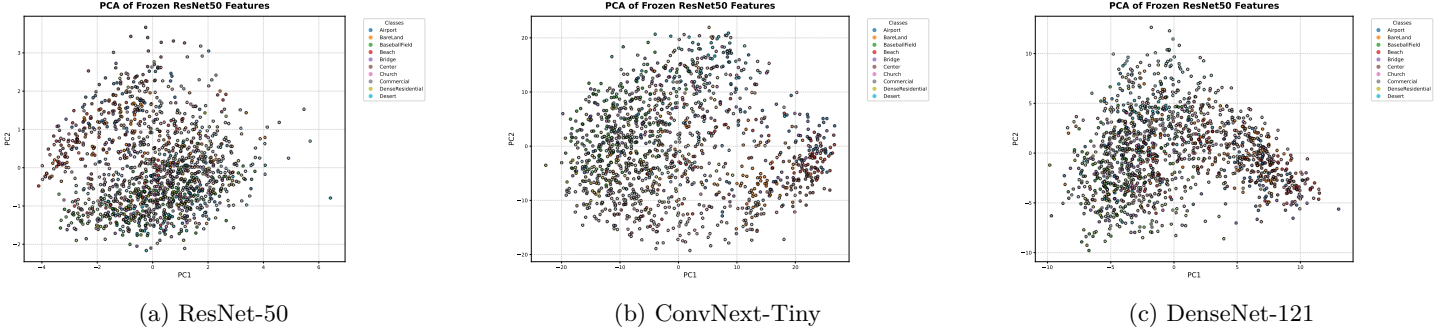


Figure 3: PCA plots for feature visualization.

were unfrozen, results of which can be seen in Figure 8. The overall performance of the backbones across these fine-tuning strategies have been consolidated in Table 3.

Table 3: Accuracy comparison across fine-tuning strategies

Strategy	ResNet50	DenseNet121	ConvNeXt-Tiny
Linear Probe	0.8011	0.8734	0.9156
Last Block	0.9026	0.917	0.951
20% Unfreeze	0.9413	0.9242	0.9592
Full Fine-tuning	0.9306	0.9185	0.7539

In addition to the accuracy variation, we also plotted the *gradient norm statistic* across layers for the different fine-tuning strategies, as seen in Figure 9 – Figure 11. Finally, we checked the **convergence stability** which is the variation of the training loss with epochs, which can be observed in Figure 13 – Figure 16.

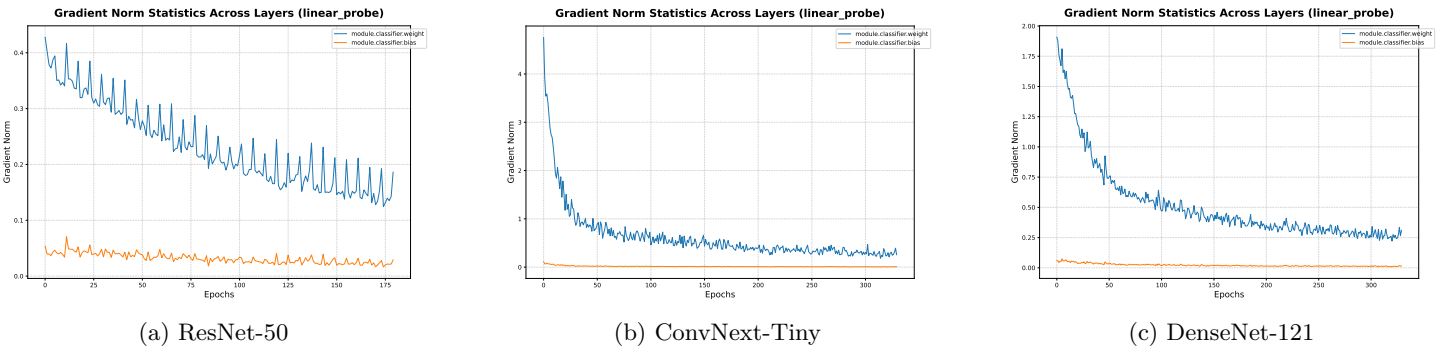
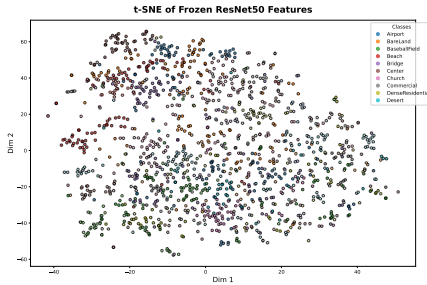
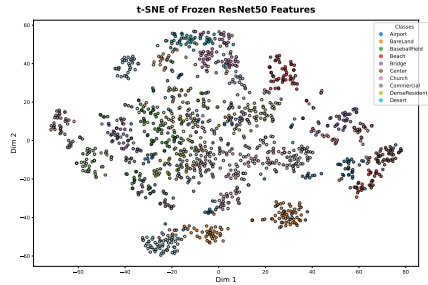


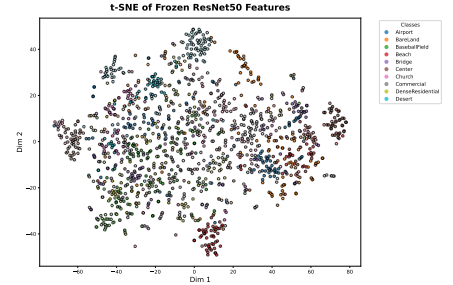
Figure 9: Gradient Norm Statistics across layers (Linear Probe)



(a) ResNet-50

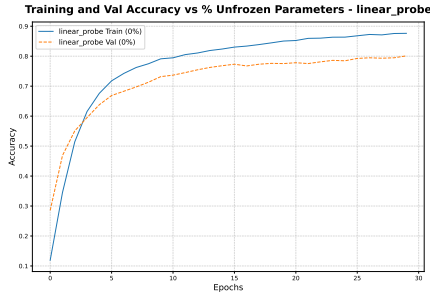


(b) ConvNext-Tiny

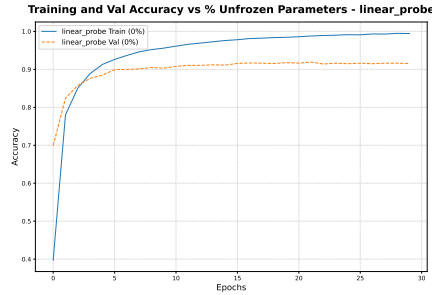


(c) DenseNet-121

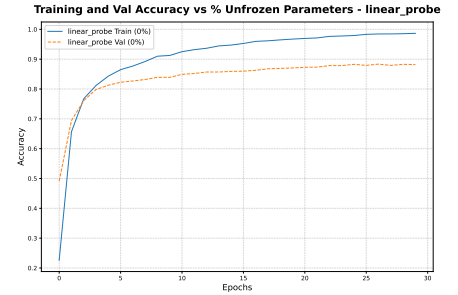
Figure 4: t-SNE plots for feature visualization.



(a) ResNet-50

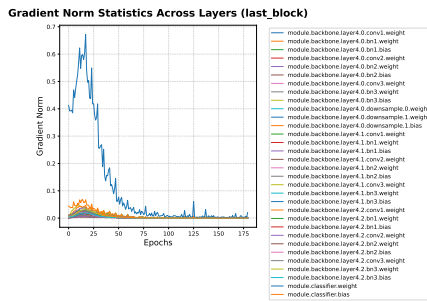


(b) ConvNext-Tiny

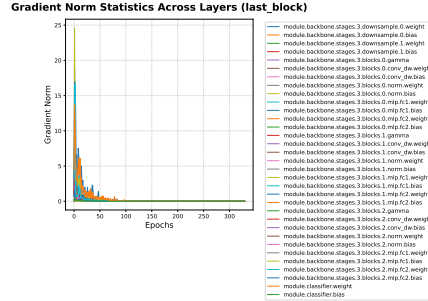


(c) DenseNet-121

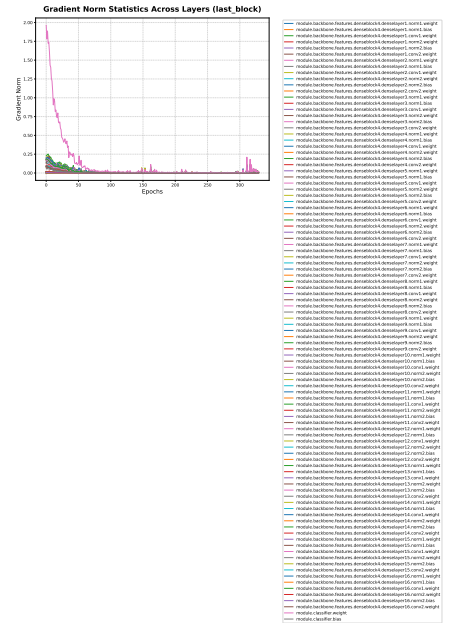
Figure 5: Training and validation accuracy with epochs (Linear probing)



(a) ResNet-50

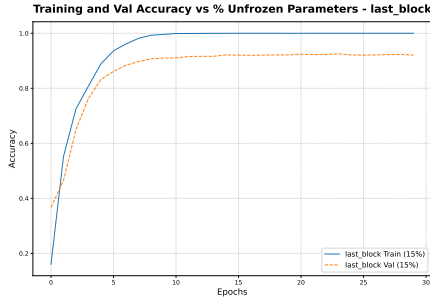


(b) ConvNext-Tiny

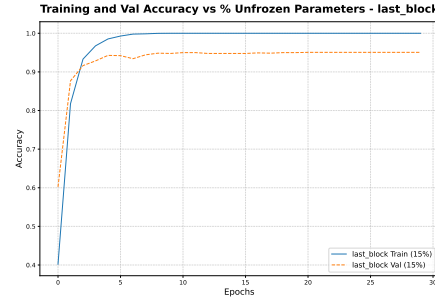


(c) DenseNet-121

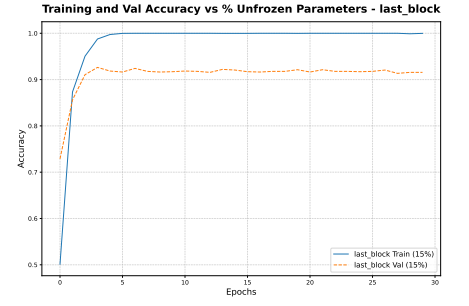
Figure 10: Gradient Norm Statistics across layers (Last block)



(a) ResNet-50

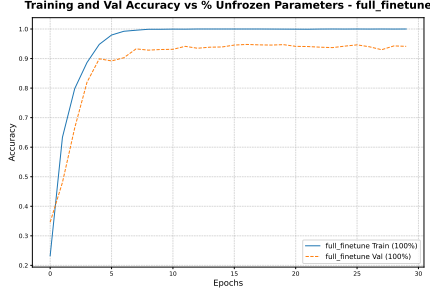


(b) ConvNext-Tiny

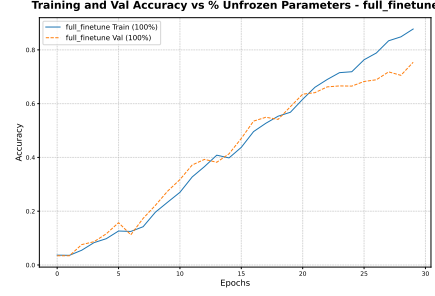


(c) DenseNet-121

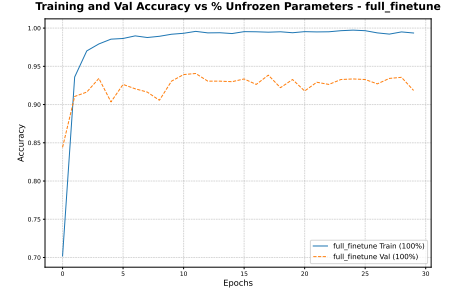
Figure 6: Training and validation accuracy with epochs (Last block + Classifier)



(a) ResNet-50

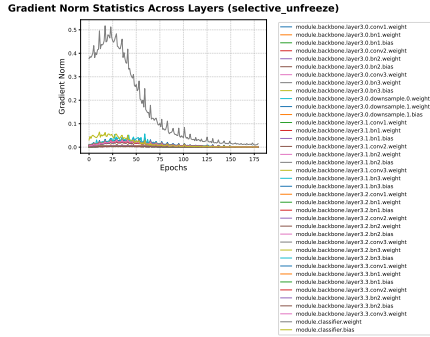


(b) ConvNext-Tiny

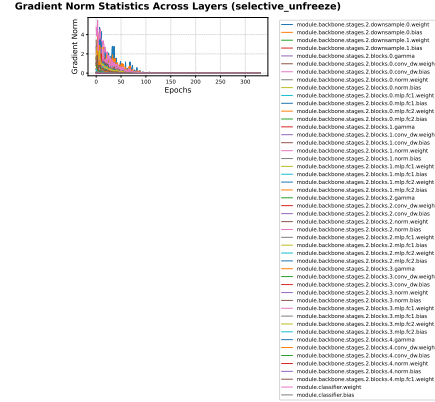


(c) DenseNet-121

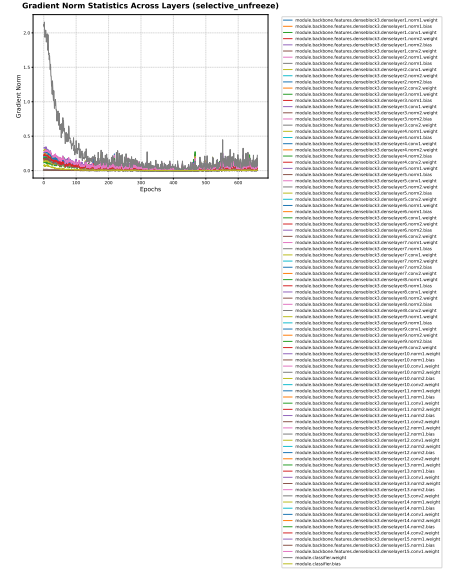
Figure 7: Training and validation accuracy with epochs (Full fine-tuning)



(a) ResNet-50

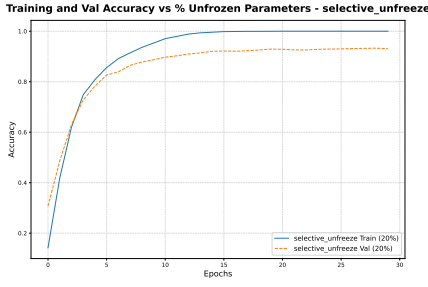


(b) ConvNext-Tiny

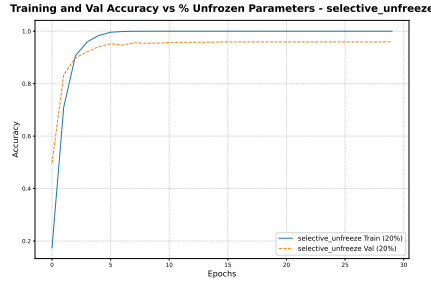


(c) DenseNet-121

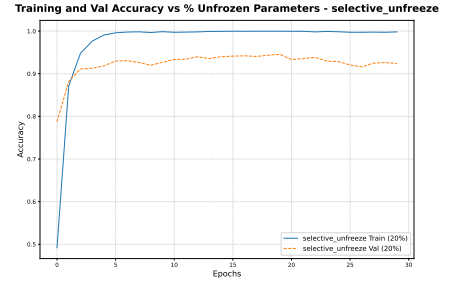
Figure 11: Gradient Norm Statistics across layers (Selective un-freezing)



(a) ResNet-50

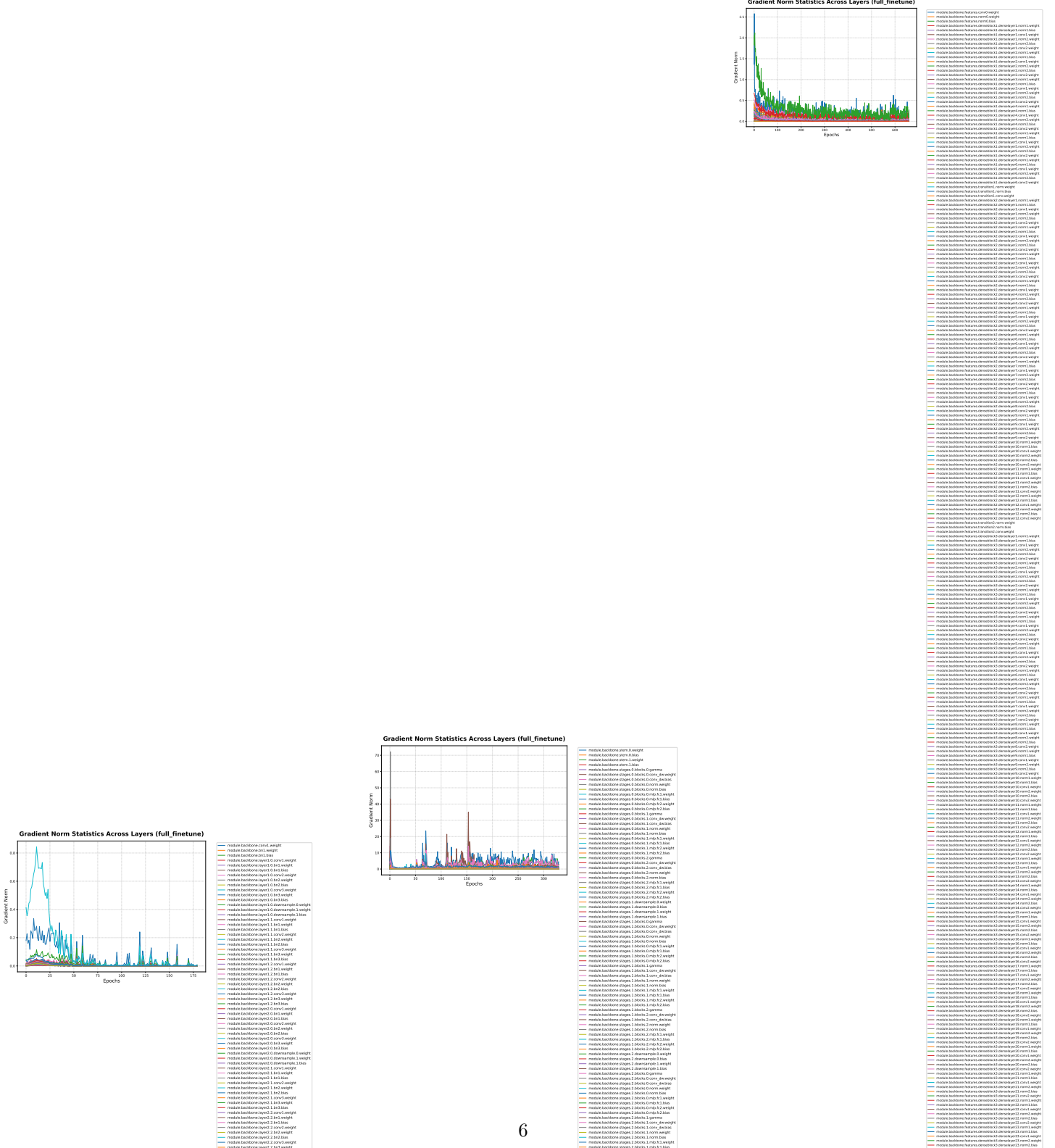


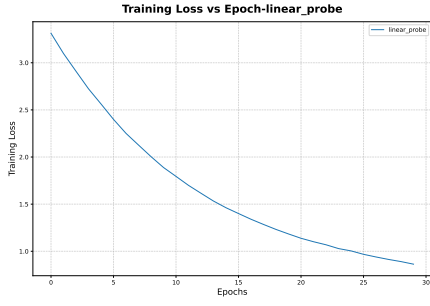
(b) ConvNext-Tiny



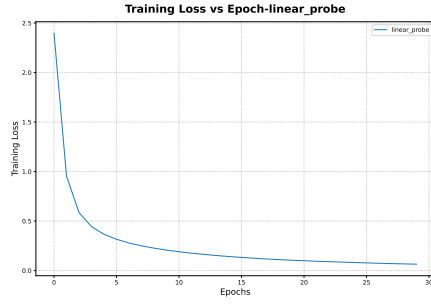
(c) DenseNet-121

Figure 8: Training and validation accuracy with epochs (Selective unfreezing)

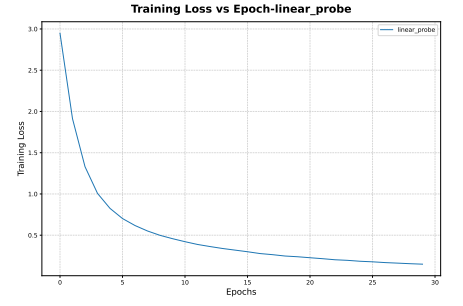




(a) ResNet-50

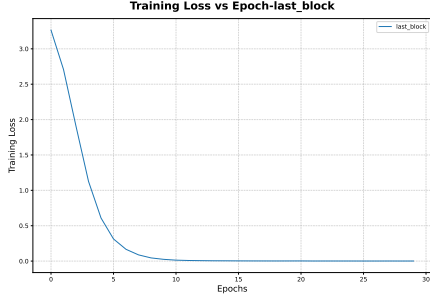


(b) ConvNext-Tiny

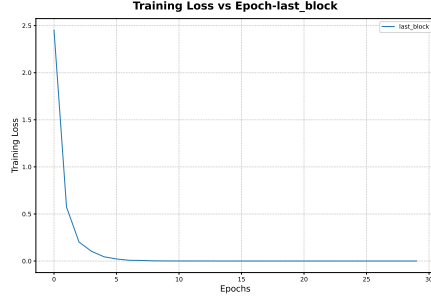


(c) DenseNet-121

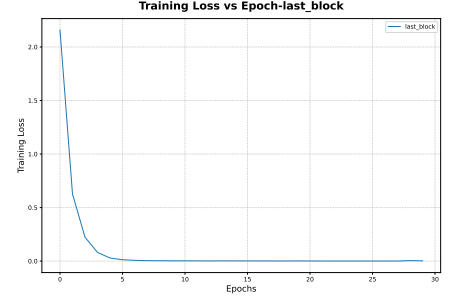
Figure 13: Convergence stability (Linear Probing)



(a) ResNet-50

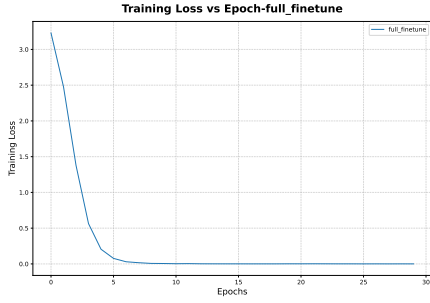


(b) ConvNext-Tiny

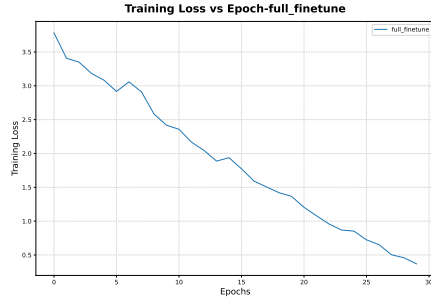


(c) DenseNet-121

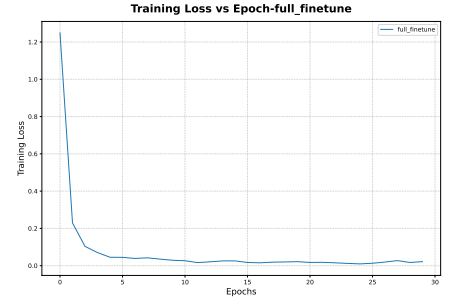
Figure 14: Convergence stability (Last block)



(a) ResNet-50

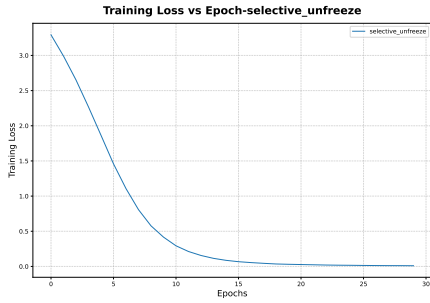


(b) ConvNext-Tiny

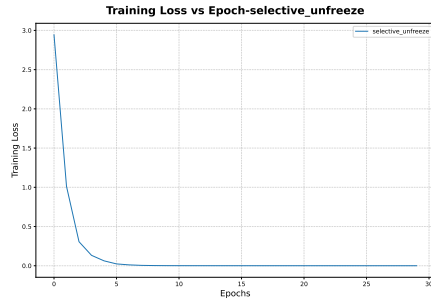


(c) DenseNet-121

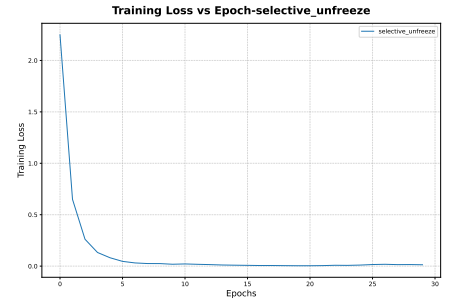
Figure 15: Convergence stability (Full fine-tuning)



(a) ResNet-50



(b) ConvNext-Tiny



(c) DenseNet-121

Figure 16: Convergence stability (Selective unfreezing)

Experiment 3 – Few-Shot Learning

We evaluated each of the backbones and their performance when trained under constrained input data. The models were trained under three regimes -

1. Train using 100% of the training data.
2. Train using 20% of the training data.
3. Train using 5% of the training data.

The random seed is kept the same to ensure reproducibility. Post-training, the models are first evaluated by checking the validation accuracy under each training regime. These are visualized as bar plots, as seen in Figure 17

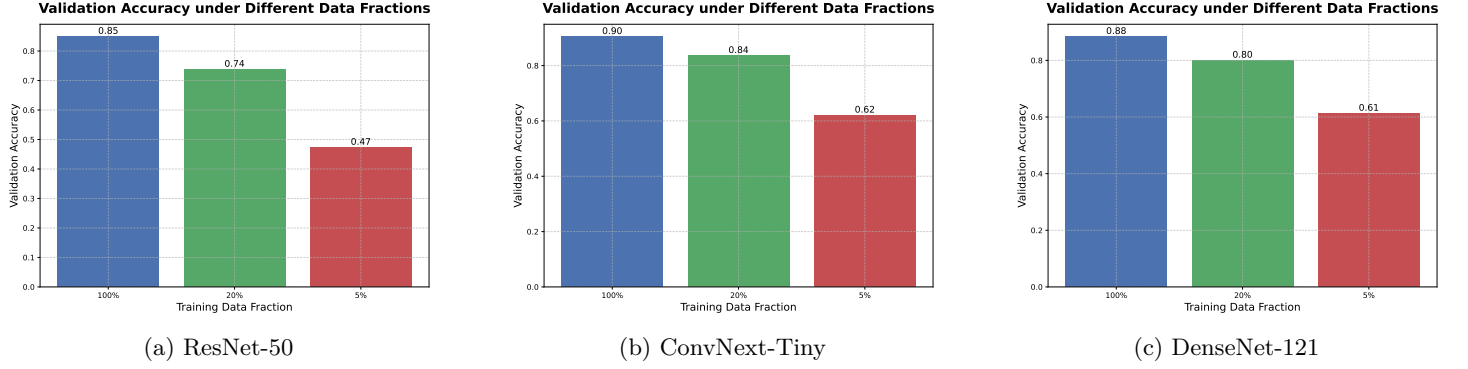


Figure 17: Validation accuracy under different training regimes.

In addition to the variation in validation accuracy, we also evaluated the models by checking the *relative performance drop* which is defined as -

$$\Delta = \frac{Acc_{100\%} - Acc_{5\%}}{Acc_{100\%}}$$

The relative performance drop along with the consolidated results for validation accuracy variation can be seen in Table 4 .

Table 4: Validation accuracy under different regimes along with relative performance drop

Model	100% Data	20% Data	5% Data	Δ
ResNet50	0.85	0.74	0.47	0.4435
DenseNet121	0.8834	0.7997	0.6123	0.3069
ConvNeXt-Tiny	0.9149	0.8627	0.6924	0.2432

Finally, we evaluated the difference between the training and validation accuracy under the three regimes. The visualization of that trend can be seen in Figure 18

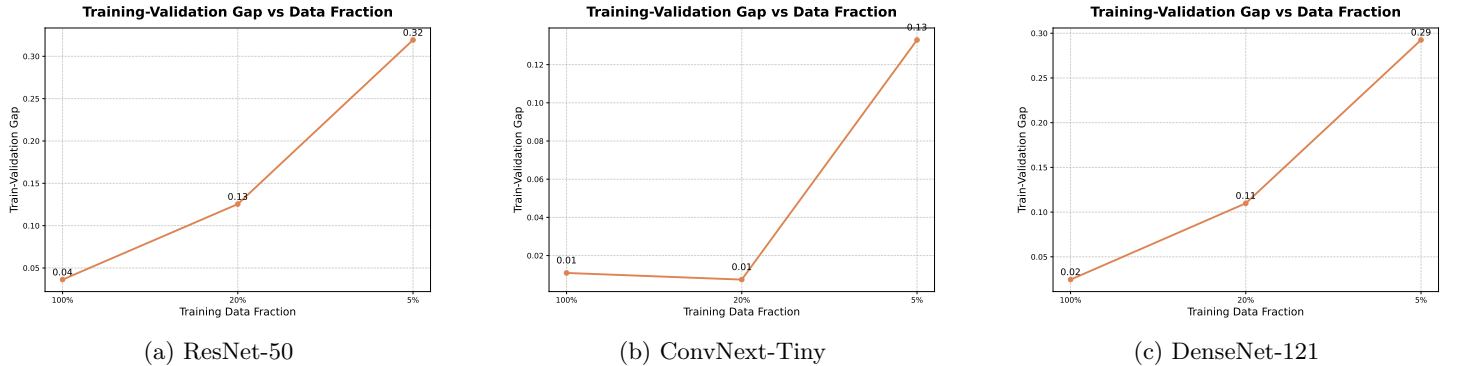


Figure 18: Training-validation accuracy gap under different training regimes.

Experiment 4 – Corruption Robustness

In this experiment, the models were evaluated under different types of input corruption and their performances were evaluated. There are **three** kinds of image corruptions that were analysed –

- Gaussian Noise ($\sigma = 0.05, 0.1, 0.2$)
- Motion Blur
- Brightness Shift

With the corrupted data¹, we first evaluated the validation accuracy of the models, as highlighted in Table 5

Table 5: Validation accuracy under corruption

Model	Gaussian ($\sigma = 0.05$)	Gaussian ($\sigma = 0.1$)	Gaussian ($\sigma = 0.2$)	Motion Blur	Brightness
ResNet50	0.0271	0.0365	0.0222	0.0486	0.0129
DenseNet121	0.0272	0.0243	0.0358	0.0308	0.0308
ConvNeXt-Tiny	0.0308	0.033	0.0215	0.0458	0.0308

Given the low validation accuracies, we proceeded to evaluate some robustness metrics as follows -

$$\text{Corruption Error} = 1 - \text{Acc}_{\text{corrupted}}$$

$$\text{Relative Robustness} = \frac{\text{Acc}_{\text{corrupted}}}{\text{Acc}_{\text{clean}}}$$

These metrics have been visualized in Figure 19.

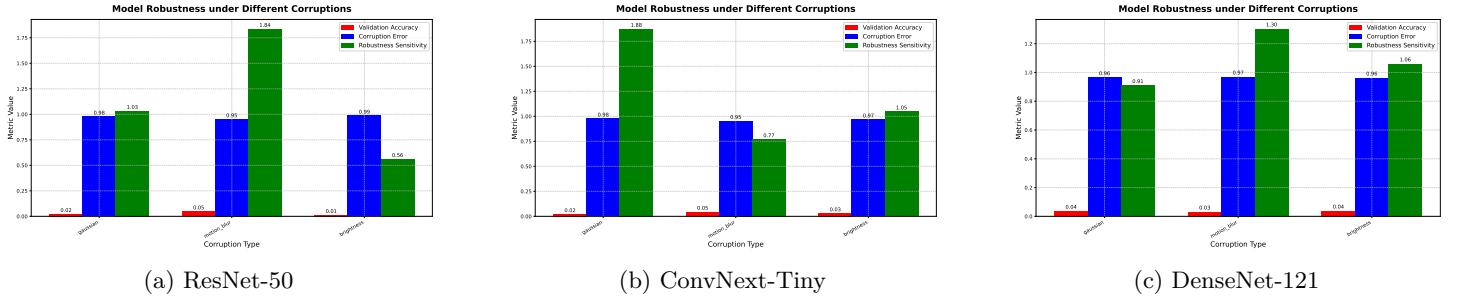


Figure 19: Corrupted accuracy and robustness sensitivity for different backbones.

Experiment 5 – Layer-wise Feature Probing

In this final experiment, we analyzed how semantic representations evolve across network depth. This was achieved by extracting intermediate representations and training separate linear classifiers using features from each depth.

Layer Selection

- Early layer
- Intermediate layer
- Final layer

Using the intermediate representations, we noted the validation accuracy, the feature norm mean and standard deviation as illustrated in Table 6

¹only at evaluation time

Table 6: Validation accuracy, feature norm mean and std at various feature layers

Model	Layer	Val. Acc.	Feature Norm mean	Feature norm std
Resnet-50	Layer 1	0.622	18.567	0.505
	Layer 3	0.93	45.086	1.36
	Layer 4	0.913	6.997	0.934
DenseNet-121	Block 1	0.618	93.36	17.56
	Block 2	0.82	28.69	1.88
	Block 3	0.92	27.75	1.911
	Block 4	0.921	28.37	0.717
ConvNeXT-Tiny	Stage 0	0.614	11.156	0.83
	Stage 2	0.933	182.115	16.19
	Stage 3	0.911	10.44	1.089

We also created a PCA 2D plot (Figure 20) across layers on a fixed subset (30 classes, 30 samples/class), using the same set of samples across all models/layers.

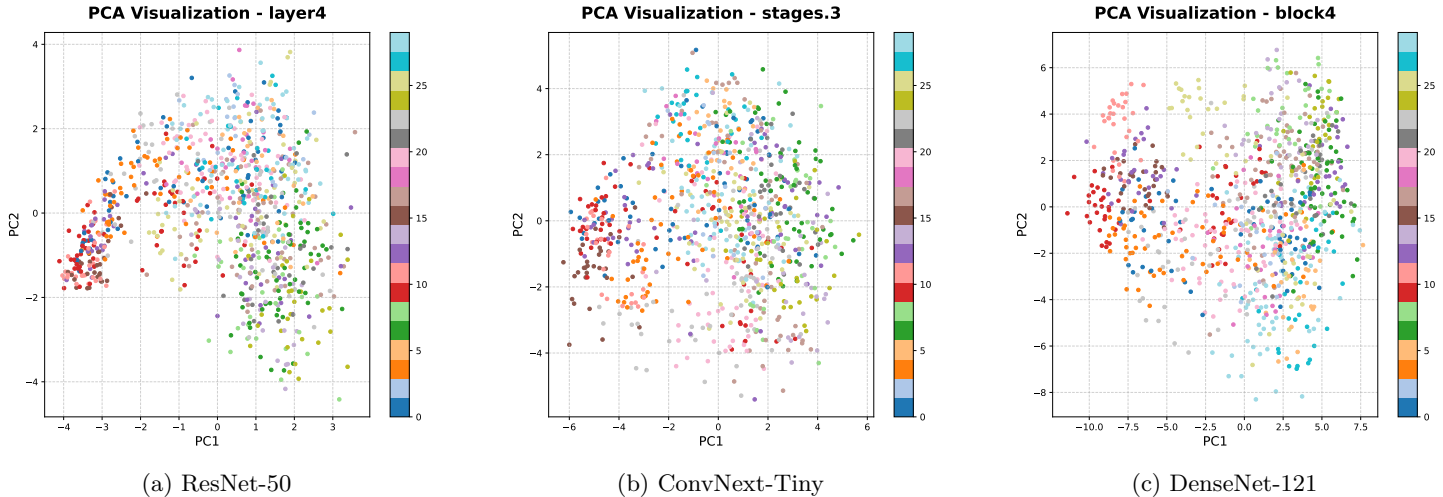


Figure 20: PCA plots for the Layer-wise features.

1 Computational Budget Report

Model	GPU Used	No of params	Total Time (Min)	Memory Usage (GB)
ResNet50	RTX A6000 ($\times 2$)	23.5M	170	15.5
DenseNet121	RTX A6000 ($\times 2$)	6.87M	118	20.5
ConvNeXt-Tiny	RTX A6000	27.2M	360	20

Table 7: Computational budget

2 Statistical Comparisons and Observations

In this section, we discuss the major insights from the experimental results of three pretrained CNN architectures: ResNet-50, ConvNeXt-Tiny, and DenseNet-121. We focus on Statistical comparisons of the representation of the different models, some unexpected findings, and other interesting results.

2.1 Statistical Comparisons

In this section, we gather insights from the feature representations of three different architectures: ResNet-50, ConvNeXt-Tiny, and DenseNet-121.

2.1.1 Insights from PCA Representation Visualizations

Fig 3 is the PCA projections that show the feature representations of the three different models.

The PCA representations visualizations of ResNet-50 in Fig. 3a show that the classes cluster into identifiable regions. Still, there is substantial overlap between the clusters, which implies that ResNet-50 learnt frozen feature representations that are only partially separable.

Visual representations of ConvNeXt-Tiny (Fig 3b) show much clearer cluster boundaries than those of ResNet-50. The overlap of the data points is noticeably lower than in ResNet-50, leading to improved separability between the different classes. The improved separability of the features of ConvNeXT-Tiny implies that the model might be learning better features than the other model architectures and should have better transferability.

Finally, the DenseNet-121 (Fig 3c) representations exhibit the highest noticeable cluster overlap and the least compact clusters, i.e., the representations look more uniform than clustered. More overlap and less compactness imply that the model struggles to distinguish some classes compared to the other model and has lower transferability.

The visualizations of the feature embeddings suggest that ConvNeXt-Tiny has better transferability compared to ResNet-50, which should have better transferability with respect to DenseNet-121.

2.1.2 Insights from t-SNE Representation Visualizations

The t-SNE emphasizes local neighborhood preservation; therefore, we focus on getting better insights into class clustering. As we will discuss in the section, we find a similar pattern in t-SNE and PCA-based comparisons of the models.

The t-SNE visual representations of ConvNeXt-Tiny (Fig 4a) show a very compact and clearly separate formation of clusters, i.e., the class clusters are grouped very tightly, and there is barely any overlap between the clusters. Therefore, the features learnt from data points of the same class lie in a similar region of the feature space, indicating that the learnt features are strong and should aid transferability.

ResNet-50 and DenseNet-121 (Fig 4b, 4c) produce t-SNE representations that are quite similar. The representations overlap more across classes and are more evenly distributed than in ConvNeXt-Tiny, hence the learnt features are weaker as well.

From the t-SNE representations, it is clear that ConvNeXt-Tiny learns stronger features compared to ResNet-50, DenseNet-121, and thus should have better transferability compared to them.

Key Takeaway: The observations from PCA and t-SNE based statistical comparisons imply that ConvNeXt-Tiny learns stronger features and thus should show better representation transferability from 2.

3 Other Observations and Insights

In this section, we discuss other observations we have made from the experimental plots.

3.1 Few-Shot Learning Behavior

In this part, we shall discuss the results few-shot learning experiment which evaluated model performance under different training data regimes (100%, 20%, and 5% of the training data).

ConvNeXt-Tiny achieves the highest validation accuracy across different situations i.e. when trained with 100%, 20%, and 5% of the training data. Moreover, ConvNeXt-Tiny shows the least amount of drop when the data is reduced (Δ in Table 4) This matches with our theory of ConvNeXt-Tiny based on **statistical comparisons** that it learns stronger features and therefore has higher transferability. ResNet-50 shows the largest performance drop when the training data is reduced which suggests that it is the least transferable model.

Furthermore, in the training-validation gap which provides insight into model generalization, ConvNeXt-Tiny the smallest training-validation gap in all regimes as well as a very small training-validation gap in both the 100% and 20% training regimes. This behavior indicates that ConvNeXt generalizes well even when part of the training data is available.

Therefore, ConvNeXt-Tiny demonstrates the strongest few-shot learning capability and the smallest training-validation gap compared to the other models hence it is the most transferable, general model among the architectures which substantiate our theory in 2.

3.2 Layer-wise Feature Probing

Layer-wise feature probing helps us to understand how representation quality changes with depth.

From the results in Table 6 we can infer that the Intermediate layers encode the most transferable features since it achieves the highest validation accuracy compared the Early and Final layers. level features show that intermediate layers achieve the highest validation accuracy compared to early and final layers. That is because Early layers capture low level features and final layers encode the specific representations for the original dataset training (in our case ImageNet) and are thus specialized whereas the Intermediate Layer doesn't suffer from both of these problems.

Therefore, the Intermediate layers turn out to be best in terms transferable semantics.

3.3 Robustness Analysis

We do corruption robustness experiment to evaluate the models invariance to perturbation to the input distribution. The perturbations can be anything from realistic perturbations to adversarial noise, however here we experiment with gaussian blur, motion blur and brightness.

In Table 5 ConvNeXt-Tiny generally (but not always) demonstrates stronger robustness compared to the other two models mostly because its learns stronger feature representations. ResNet-50 and DenseNet-121 shows quite stable behavior w.r.t. different corruption types even though their robust accuracy is generally lower than ConvNeXt-Tiny.

Robustness and validation accuracy are generally at a trade-off since improving validation accuracy requires the model to get specialized to the original input distribution but reduces the ability to generalize when the input distribution changes. Therefore, this is the reason fine-tuning to improve test accuracy leads to weaker robustness.

Overall, ConvNeXt-Tiny emerges as the strongest backbone for aerial scene classification due to its superior representation learning capability and strong few-shot performance, although it exhibits slightly less stable convergence behavior during full fine-tuning.

3.4 Unexpected Finding: Training–Validation Gap Behaviour in ConvNeXt-Tiny

An interesting and somewhat unexpected observation was found in the training–validation gap analysis shown in Figure 18(b), in the case of ConvNeXt-Tiny. The training–validation gap for both the **100%** and **20%** training data regimes are almost zero.

In case of ResNet-50 and DenseNet-121, the training–validation gap increases as training data the goes from 100% to 20% to 5%. However, in this case the gap remains almost unchanged when moving from the full dataset to the 20% regime, and even when the dataset is reduced further to **5%** the validation increases but much lower compared to ResNet-50 and DenseNet-121.

The plausible explanation is the strong feature representation learnt by ConvNeXt-Tiny when pretrained on ImageNet allows it to be very transferable. The pretrained features might be general enough that it works well with the aerial scene classification task with little data as well. As a result, with only a fraction of the training data (20% of the training data) it is able to outperform the other models with 100% of the data.

Therefore, this unexpected observation indicates that ConvNeXt-Tiny demonstrates strong generalization ability which we discussed in earlier sections.

3.5 Failure Case

A failure case is observed in 7b and 15b, where ConvNeXt-Tiny doesn't converge during full fine-tuning unlike ResNet-50 and DenseNet-121. Its training loss decreases smoothly across epochs, ConvNeXt but never converges to a minima.

Two possible reasons are as follows:

- ConvNeXt architectures requires more careful hyperparameter tuning since it implements optimization techniques like AdamW, label smoothing tools to optimize which er aren't using. As a result, the optimizer may be in a unstable parameter region which prevents smooth convergence.
- Moreover, it could be that its large representational capacity ConvNeXt-Tiny is working against it. Fine-tuning all parameters trained to ImageNet to AID (Aerial Image Dataset) can lead to large shifts in the pretrained feature space which would require a lot more of pretraining..

3.6 Conclusions

Following are the overall conclusions we have gathered from the experiments and its analysis of this report:

- ConvNeXt-Tiny learns the most strong and therefore transferable feature representations. That's why it is ahead of ResNet-50 and DenseNet-121 in our results for Few-Shot Learning and Robustness.
- Intermediate network layers provide the most transferable representations.
- Finetuning reduces Robustness since it leads the model to learn specialized features for the original input distribution.

In this report, the results show representation transfer of models, fine-tuning capabilities, and robustness of pretrained convolutional networks adapting to new visual recognition tasks. The experiments reveal that architectural design, layer selection, and fine-tuning all play a significant role in determining how effectively pretrained models can generalize.